

AI Detector report by



8/13/2026

Model v7.1.0 

6,245 words

19%

of text is likely AI

AI-generated		19%
Human-written & AI-refined		0%
Human-written		81%

Caution: Our AI Detector is advanced, but no detectors are 100% reliable, no matter what their accuracy scores claim. Never use AI detection alone to make decisions that could impact a person's career or academic standing.

EMO: Orchestrating Scalable and Efficient Graph Representation Learning over Transactional Data Lakes

Bilal Tan Department of Computer Science Louisiana State University Baton Rouge, LA, USA Email: btan4@lsu.edu
Kisung Lee Department of Computer Science Louisiana State University Baton Rouge, LA, USA Email: klee76@lsu.edu

Abstract—Distributed Graph Representation Learning (GRL) on massive-scale graphs is bottlenecked by the neighborhood explosion problem: computing a single node embedding requires recursively aggregating features from an exponentially growing set of neighbors, forcing existing frameworks to choose between costly synchronized cross-worker communication and training in isolation on decoupled partitions. This paper presents EMO (Executor-level Multi-community Orchestrator), a distributed systems framework that orchestrates decoupled GRL workloads natively on transactional data lakes. EMO models graph partitioning, boundary detection, and

community-aware auxiliary graph construction as standard relational operators—joins, group-bys, and broadcasts—executed through Apache Spark SQL on Delta Lake over Amazon S3. To recover the structural context lost by partitioning, EMO implements the CAAN (Community-Aware Auxiliary Network) super-node abstraction as a relational plan, compressing communities into centroid-feature super-nodes and distributing them to executors via Spark broadcast variables. The framework leverages Delta Lake's ACID transaction logs for experiment-level isolation and phase-level checkpoint reuse, implements dynamic EBS storage redirection on AWS EMR workers to prevent root-volume out-of-memory crashes, and utilizes Apache Arrow for zero-copy JVM-Python data transfer. Experimental evaluation on graphs ranging from 11K to 111M nodes demonstrates that EMO recovers up to 97.8% of full-graph accuracy with zero cross-worker training communication, and scales SIGN-style propagation to the ogbn-papers100M graph (3.2B edges) on commodity CPU clusters. All source code and reproducibility benchmarks are available at <https://github.com/bilaltan/emo-pipeline>.

Index Terms—Data Lakes, Graph Representation Learning, Distributed Systems, Apache Spark, Delta Lake, AWS EMR.

1. Introduction

Graph Neural Networks (GNNs) [1], [2] have become the dominant paradigm for learning on non-Euclidean graph data, yet scaling them to web-scale graphs remains a fundamental systems challenge. The root cause is the neighborhood explosion problem: computing a node's representation requires recursively aggregating features from multi-hop neighborhoods, so the per-node working set grows exponentially with depth. Distributed frameworks such as DistDGL [3] address this by synchronizing node features and gradients across workers through parameter servers, but the resulting network traffic and infrastructure costs are prohibitive for many organizations. An alternative is community-decoupled training. Partitioning the graph

into dense communities—via Louvain [13] or Label Propagation [14]—enables each worker to train an independent GNN with zero cross-worker communication. However, this decoupling introduces two well-documented accuracy bottlenecks. First, cutting inter-community edges causes boundary accuracy degradation: nodes at partition boundaries lose topological context from neighboring communities. Second, graph community sizes follow a power-law distribution, so numerous small (“minor”) communities provide insufficient training samples, leading to overfitting. Recent theoretical work on Community-Aware Auxiliary Networks (CAAN) [7] and multi-level GRL [8], [9] proposes constructing auxiliary graphs in which major communities are compressed into super-nodes (represented by feature centroids) to recover macro-level context, while minor-community nodes are aggregated globally to alleviate sample scarcity. Despite their theoretical promise, these formulations lack a scalable, distributed systems realization. Running such workflows today requires bespoke ETL pipelines that manually export graphs from data warehouses to deep learning environments—an error-prone, non-reproducible process. This paper presents EMO (Executor-level Multi-community Orchestrator), a distributed systems framework that bridges transactional data lakes and GRL training. EMO models the entire graph mining and GRL pipeline—from ingestion through partitioning, boundary detection, super-node construction, and training—as relational queries executed on Delta Lake [11] over Amazon S3 via Apache Spark SQL. Figure 1 illustrates the physical deployment architecture; Figure 2 shows the logical phase pipeline. The contributions of EMO are: 1) Data Lake-Native GRL Storage (C1). EMO unifies graph storage and ML execution on transactional Delta Lake tables, providing ACID experiment isolation, phase-level checkpoint reuse via transaction logs, and schema-enforced reproducibil-

ity (Section 3.5).

2)Relational Graph Operators (C2). Graph partitioning, boundary detection, and CAAN's auxiliary graph generation [7] are formulated as standard Spark SQL joins and group-by aggregations, eliminating the need for specialized graph-processing infrastructure (Section IV).

3)Cloud-Native Runtime Hardening (C3). Dynamic EBS volume redirection prevents root-disk OOM on EMR workers, YARN-mediated library synchronization ensures executor consistency, and Apache Arrow enables zero-copy JVM-Python data transfer (Section V).

4)Empirical Verification. Evaluation on graphs from 11K to 111M nodes demonstrates that EMO recovers up to 97.8% of full-graph accuracy with zero training-time communication, and scales SIGN-style propagation to the ogbn-papers100M graph (111M nodes, 3.2B edges) on commodity CPU clusters (Section VI).

2. Related Work

Distributed GNN Training. Frameworks such as DistDGL [3], PyG-Distributed [6], and Marius [5] scale GNNs through neighborhood sampling, synchronized feature fetches, or storage-centric caching. Serverless systems such as Dorylus [4] decouple compute into fine-

grained lambda threads while managing graph partitions over network storage. While effective, these systems still require dedicated synchronization infrastructure or specialized serverless runtimes. EMO takes the complementary approach: it executes natively on enterprise transactional data lakes (Delta Lake over Amazon S3) with Apache Spark, combining relational partitioning with cached SIGN-style feature propagation [10] to achieve zero inter-worker training communication.

Communication-Reduced and Federated GNN Training. GraphSAINT [18] builds localized mini-batches through subgraph sampling but still assumes shared-memory access to the full graph. FedSage+ [19] handles missing cross-client neighbors in federated settings by training local neighbor generators, yet requires iterative client-server communication rounds. EMO eliminates all training-time communication by recovering cross-partition context through a one-time relational super-node construction step.

Scalable Pre-Computation. SIGN [10] decouples feature extraction from classification by pre-computing multi-hop aggregated features, enabling edge-free downstream training. Extensions such as GAMLP [20] and C&S [21] add attention-weighted hop aggregation and label propagation post-processing, respectively. EMO's Phase 3.7/3.8 path follows the same design philosophy but contributes a

TABLE 1. CORE NOTATION USED IN SECTIONS III–V.

SymbolMeaning

$G = (V, E, X)$ Attributed graph (nodes, edges, features)

$\pi(v)$ Community assignment of node v

$b(v)$ Boundary indicator for node v

K Major/minor community threshold

T

total End-to-end pipeline runtime

distributed systems realization: each propagation hop is executed as a Spark vector-mean aggregation and checkpointed in Delta Lake, making the entire pipeline resumable and parallelizable.

Graph Partitioning. METIS [15] produces balanced cuts but is computationally expensive on massive graphs.

Label Propagation (LPA) [14] runs natively in Spark, while Louvain [13] yields higher-quality structural communities at the cost of driver memory. EMO supports pluggable partitioners; our evaluation uses both LPA and Louvain.

Transactional Data Lakes for ML. Delta Lake [11], Apache Hudi, and Apache Iceberg provide ACID transactions, time-travel queries, and schema enforcement over cloud object stores. Lakehouse architectures have been adopted for tabular ML workloads, but no prior system has applied transactional storage contracts to distributed GRL. EMO is the first framework to enforce experiment isolation, phase-level checkpointing, and metadata versioning specifically for GNN training on a data lake.

3. EMO Model and System Architecture

3.1. Problem Setup and Design Goals

Let $G = (V, E, X)$ denote an attributed graph with node set V , edge set E , and node features $X \in \mathbb{R}^{|V| \times d}$.

A community partitioner $\pi: V \rightarrow \{1, \dots, m\}$ assigns each node to one of m communities. The boundary indicator

$$b(v) = \mathbb{1}_{\exists u \in N(v) : \pi(u) \neq \pi(v)}$$

identifies nodes that lose at least one edge when the graph is split along community boundaries. EMO targets a joint systems objective: minimize synchronization cost while preserving global structural context for boundary and minor-community nodes. This objective decomposes into three design goals: (i) communication-free local compute during training-critical stages, (ii) relational recovery of cross-community context via super-node abstractions, and (iii) transactional reproducibility through versioned, schema-enforced storage. Table 1 summarizes the primary mathematical notation used across Sections III–V.

3.2. Physical Infrastructure and Deployment

Figure 1 depicts EMO's physical deployment on an AWS EMR cluster. The system comprises a single Spark driver node that orchestrates pipeline phases and dispatches work to 8 worker nodes via YARN. Each worker runs multiple Spark executors, each of which contains a JVM heap partition (for Spark SQL relational operations) and an off-

heap allocation (for PyTorch and DGL tensor computation).

Apache Arrow provides zero-copy JVM–Python serialization, avoiding the overhead of pickle-based data transfer.

All persistent state—graph tables, partitioned subgraphs, propagated features, and experiment metadata—resides in Delta Lake tables on Amazon S3, accessed through the Delta Lake protocol.

3.3. Two-Branch Execution Model

EMO exposes two logical execution branches over a shared transactional storage backbone (Figure 2). Branch A is the primary evaluation path: Phase 0 ingests raw graph data into Delta Lake; Phase 3.7 computes multi-hop SIGN-style feature propagation and caches each hop as a separate Delta table; Phase 3.8 trains a global edge-free classifier on the concatenated propagated features; and Phase 5 generates reporting artifacts. Branch B executes community-decoupled training: Phase 1 runs community detection (LPA or Louvain), Phase 2 extracts partitioned subgraphs with boundary metadata, and Phase 3/3b trains per-community GNNs with optional CAAN super-node augmentation. Both branches share identical storage contracts and checkpoint semantics, enabling controlled comparisons without rewriting orchestration logic.

3.4. Transactional Experiment Isolation

Delta Lake serves as a first-class systems component rather than a passive storage layer. EMO enforces an

experiment-level namespace contract that encodes the experiment identifier, dataset, and partitioning algorithm into each table's S3 path:

Path

P2 = delta-data/{dataset}/phase2_nodes/{expid}_{dataset}_{alg}
(2)

This naming convention ensures that distinct runs—varying in experiment ID, model configuration, or partitioning algorithm—can execute concurrently without write collisions. Phase-level resumability is governed by transactional evidence: when EMO detects a valid commit in a table's `_delta_log/` directory, it reuses that checkpoint and skips recomputation. This converts expensive graph preprocessing from mandatory repeated work into a deterministic, cacheable stage.

3.5. Delta Lake Storage Schema

Table 2 summarizes the Delta Lake table schemas that EMO materializes across pipeline phases. All tables are stored as Parquet files on S3 with Delta Lake transaction logs providing ACID semantics.

Phase 0 (Ingestion) loads raw graph data from OGB or DGL, symmetrizes directed edges for GNN message passing, and writes three foundational tables: `nodes/` (node features and labels), `edges/` (undirected edge pairs), and `masks/` (train/validation/test split assignments from OGB). These tables are written once and shared across all down-

stream experiments.

Phase1(CommunityDetection)producesa

communities/{alg}/tablemappingeachnode

ID to its community ID. The table is keyed by the

partitioning algorithm (e.g., lpa or louvain), allowing

multiple algorithms to coexist without conflict.

Phase 2 (Subgraph Extraction) materializes two ta-

bles per experiment configuration. The phase2_nodes

table augments each node with its community assign-

ment and a boolean is_boundary flag, computed by

comparing a node's original degree against its intra-

community degree: $\text{is_boundary}(v) \Leftrightarrow \text{deg}$

$\text{orig}(v) >$

deg

$\text{intra}(v)$. The phase2_edges table retains only

intra-community edges, tagged with the community ID

for partition-local reads. Both tables are physically

clustered on S3 using `repartition (N, 'community`

`id')`

and `sortWithinPartitions ('communityid')` to achieve S3

block-level locality when executors read their assigned com-

munities.

Phase 3.7 (Feature Propagation) materializes K hop-

level tables, each storing the mean-aggregated features from

the previous hop. The final table concatenates all hops

[x (0)

$\|x(1)$

$\| \dots \|x(K)$

] into a single feature vector per node.

Each hop is checkpointed independently, enabling partial-failure recovery without recomputing earlier hops.

3.6. Partition Contracts for Community Workloads

Phase 2 materializes two explicit data contracts per community: (i) intra-community subgraph artifacts for local model execution and (ii) boundary-node metadata for cross-community context recovery. Separating these concerns allows EMO to schedule high-throughput local computation independently from lower-frequency global-context construction, simplifying both workload placement and failure recovery.

4. Relational Formulation for Community-Aware GRL

A central technical contribution of EMO is expressing graph-native operations—boundary detection, super-node construction, and feature propagation—as relational plans with explicit execution semantics over Delta Lake tables.

4.1. Graph-to-Relational Encoding

EMO represents the graph as three normalized Delta tables: nodes/ (features and labels), edges/ (directed pairs), and communities/ (node-to-partition mapping).

Boundary discovery becomes a join-and-aggregate plan rather than a bespoke graph traversal. Concretely, a node

is classified as boundary when its original degree exceeds
its intra-community degree:

is

$\text{boundary}(v) \Leftrightarrow \text{deg}$

$\text{orig}(v) > \text{deg}$

intra (v)(3) AWS EMR Cluster (Compute Plane) • emr-7.x • Apache Spark 3.5 • Delta Lake
3.2 • Apache YARN Resource Manager

Amazon S3 (Transactional Data Lake Storage Plane) Root URI: s3://us-east-1-s3-gnn/delta-
data/{dataset}/ Master / Driver Node (m5.2xlarge • 8 vCPUs • 32 GB RAM)

YARN Resource Manager Executor scheduling & container lifecycle

Spark Driver (PySpark) • Global Phase Orchestration (Phases 0-5) • Broadcast centroid /
super-node tables

igraph / Local Detection Phase 1: Louvain community partitioning (collected graph
topology in driver memory)

Driver Local Storage • /mnt/tmp (Dynamic redirection) • Matplotlib / LaTeX report output

Apache Arrow Zero-Copy Data Transfer PyArrow RecordBatches JVM Heap $\Leftrightarrow$ PyTorch

Worker 1 (m5.4xlarge • 64 GB)

Spark Executor 1 Spark Executor 2

Local Attached EBS Volume HOME & TMPDIR Redirect • PyTorch ~/.cache & wheels •

Prevents root 15GB OOM (F2) Worker 2 (m5.4xlarge • 64 GB)

Spark Executor 3 Spark Executor 4

Local Attached EBS Volume HOME & TMPDIR Redirect • PyTorch ~/.cache & wheels •

Prevents root 15GB OOM (F2) Worker 3 (m5.4xlarge • 64 GB)

Spark Executor 5 Spark Executor 6

Local Attached EBS Volume HOME & TMPDIR Redirect • PyTorch ~/.cache & wheels •

Prevents root 15GB OOM (F2) • • • Worker 8 (m5.4xlarge • 64 GB)

Spark Executor 15 Spark Executor 16

Local Attached EBS Volume HOME & TMPDIR Redirect • PyTorch ~/.cache & wheels •

Prevents root 15GB OOM (F2) Cluster Total: 8 Worker Nodes • 16 Executors • 64 vCPUs •

512 GB Cluster RAM

_delta_log/ ACID Transaction Plane Per-experiment isolation • Zero write conflict •

Checkpoint reuse

nodes/ Raw / Ingested Nodes

Δ Delta Lake (Parquet + Log) edges/ Undirected Symmetrized Edges

Δ Delta Lake (Parquet + Log) masks/ Evaluation Splits

Δ Delta Lake (Parquet + Log) communities/{alg}/ Community Partitioning Map

Δ Delta Lake (Parquet + Log) phase2_nodes/{tag}/ Partitioned Subgraphs + Meta

Δ Delta Lake (Parquet + Log) phase37_prop./hop_k/ Multi-Hop Propagated Features

Δ Delta Lake (Parquet + Log) YARN Executor Launch & Heartbeats

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

JVM Heap 28 GB Spark SQL Joins/Filter Off-Heap 12 GB Ovhd PyTorch/DGL Tensors/Train

Phase 0

id: LONG (PK) label: INT features: ARRAY<FLOAT> Phase 0

src: LONG (FK) dst: LONG (FK) Phase 0

id: LONG (FK) split: STRING ('train','val','test') Phase 1

id: LONG (FK) community_id: LONG (LPA / Louvain) Phase 2

id, label, features split, community_id is_boundary: BOOL Phase 3.7

id: LONG (FK) features: ARRAY<FLOAT> (Cached SIGN hops 1..K) Delta Lake Protocol (Spark SQL $\rightleftharpoons$ S3 Object Store) ACID Transactions • Schema Enforcement • Repartition Locality • Checkpoint Skip

Figure 1. EMO physical deployment architecture on AWS EMR. The Spark driver coordinates 8 m5.4xlarge worker nodes via YARN. Each worker runs two Spark executors with JVM heap (relational SQL) and off-heap (PyTorch/DGL) memory regions, plus a local EBS volume for temporary storage. All persistent data is stored as Delta Lake tables on Amazon S3, accessed through the Delta Lake transactional protocol with ACID isolation and checkpoint reuse.

EMO Logical Pipeline Architecture & Two-Branch Execution Model • Decoupled GRL • Relational Graph Algebra • Zero Cross-Worker Training Traffic

Delta Lake Tables & Partition Contracts (S3)URI: s3://us-east-1-s3-gnn/delta-data/{dataset}/ Core Technical Contributions Branch A: Full-Graph Propagation & Classification (Primary Path)

- OGB / DGL raw data load • Undirected edge symmetrization • Materialize nodes, edges, masks • K-hop mean-pooling via Spark SQL • Summarizer.mean JVM vector ops • Materialize cached hop tables $[x^0 \dots x^K]$ • LogisticRegression / MLP probe • Fits on concatenated features • 100% full-graph node coverage • Multi-dataset metric aggregation • Test accuracy / AUC evaluation • Automated LaTeX & plot generation

Branch B: Community-Decoupled GNNs & CAAN Super-Node Path

- Spark native LPA (scalable) • igraph Louvain (driver memory) • Materialize communities/{alg}/ • Boundary tag: $\text{deg_orig} > \text{deg_intra}$ • Intra-community edge filtering • repartition('community_id') S3 locality • Independent GraphSAGE / GATv2 • Trained in executor worker memory • Zero cross-worker network traffic • Major community centroid super-nodes • Broadcast (S, M) auxiliary context • Recovers 97.8% boundary accuracy

Base Graph (P0) nodes/ edges/ masks/

id, label, features src, dst id, split Δ Delta Parquet Communities (P1) communities/ {alg}/

id: LONG community_id: LONG Δ Delta Parquet Subgraphs (P2) phase2_nodes/

phase2_edges/

id, features, is_boundary src, dst, comm_id Δ Delta Parquet Propagated (P3.7) hop_k/

features_kk/

hop 1..K mean cache $[x^0 \parallel x^1 \parallel \dots \parallel x^K]$ Δ Delta Parquet C1: Transactional Lakehouse Storage

ACID experiment isolation, phase checkpoint reuse via `_delta_log`, schema enforcement.

C2: Relational Graph Operations Boundary detection & CAAN auxiliary super-node

construction via Spark SQL joins/aggregations. C3: Cloud-Native Systems Hardening

Dynamic EBS redirection (F2), YARN overhead sizing (F1), and Arrow zero-copy

serialization. Phase 0 : Graph Ingestion Phase 3.7 : SIGN Feature Propagation Phase 3.8 :

Global Edge-Free Classifier Phase 5 : Evaluation & Reporting

Phase 1 : Community Detection Phase 2 : Relational Subgraph Extraction Phase 3 :

Decoupled Community GNNs Phase 3b : CAAN Super-Node Recovery

Shared Delta Lake Transactional Storage Plane on Amazon S3 ACID Isolation • Checkpoint

Reuse via `_delta_log` • Schema Enforcement • Parquet Format

Figure 2. EMO logical pipeline architecture. Branch A (Phase 0 $\rightarrow$ 3.7 $\rightarrow$ 3.8 $\rightarrow$ 5) is the primary evaluation path; Branch B supports partition-centric ablations. Both branches share a Delta Lake transactional plane on S3. Labels C1–C3 map pipeline stages to the three technical contributions.

where `deg orig` is computed from the full edge table and

`deg intra` from edges filtered to matching community IDs at

both endpoints. Both counts are single group-by aggrega-

tions. 4.2. Boundary Recovery via Super-Node Abstrac-

tion

For each major community C_i (size $\geq K$), EMO compresses every other major community C_j ($j \neq i$) into a single super-node with centroid feature:

$$f_{\text{super}}(C_j) = \frac{1}{|C_j|} \sum_{v \in C_j} f_v$$

(4) TABLE 2. DELTA LAKE TABLE SCHEMAS USED BY EMO ACROSS PIPELINE PHASES. ALL TABLES ARE STORED ON AMAZON S3 WITH DELTA LAKE TRANSACTION LOGS (_DELTA_LOG/) PROVIDING ACID SEMANTICS, SCHEMA ENFORCEMENT, AND CHECKPOINT REUSE. COLUMN TYPES FOLLOW SPARK SQL CONVENTIONS.

Phase	Table	Path	Schema (column : type)	Description
Phase 0	nodes/		id:LONG, label:INT, features:ARRAY<FLOAT>	Symmetrized node features and ground-truth labels
Phase 0	edges/		src:LONG, dst:LONG	Symmetrized undirected edge connectivity
Phase 0	masks/		id:LONG, split:STRING	Standard train/validation/test split assignment
Phase 1	communities/{alg}/		id:LONG, community_id:LONG	Node-to-community mapping from LPA / Louvain
Phase 2	phase2_nodes/{tag}/		id:LONG, label:INT, features:ARRAY<FLOAT>, split:STRING, community_id:LONG, is_boundary:BOOL	Partitioned nodes annotated with community ID and boundary indicator flag
Phase 2	phase2_edges/{tag}/		src:LONG, dst:LONG, community_id:LONG	Filtered intra-community edges for local GNNs
Phase 3.7	phase37_.../hop_k/		id:LONG, features:ARRAY<FLOAT>	k-th hop mean-aggregated features (cached)
Phase 3.7	phase37_.../features_kK/		id:LONG, label:INT, split:STRING, features:ARRAY<FLOAT>	Concatenated SIGN-style representation $[x^{(0)} \parallel \dots \parallel x^{(K)}]$

TABLE 3. RELATIONAL WORKFLOW: BOUNDARY DETECTION AND SUPER-NODE

CONSTRUCTION.

Input: node table N , edge table E , partition map π , threshold K Output: boundary-node table B , super-node table S , minor-node table M

1. Join E with π on both source and destination endpoints. 2. Identify edge-crossing events where $\pi(\text{src}) \neq \pi(\text{dst})$. 3. Aggregate crossing counts per node; emit boundary set B . 4. Compute community sizes; split into major ($|C| \geq K$) and minor ($|C| < K$). 5. For each major community, compute feature centroid to produce super-nodes S . 6. Retain minor-community nodes as explicit entries in M . 7. Broadcast (S, M) to executor-side training tasks alongside local partition data.

Minor communities ($|C| < K$) are retained as explicit nodes to avoid information loss from over-compression.

The resulting auxiliary context set—super-nodes plus minor-community nodes—is distributed to all executors through Spark broadcast variables, giving each local training task a consistent global sketch without introducing synchronous cross-worker message passing during training.

4.3. Algorithmic Execution Semantics

Table 3 summarizes the relational workflow for boundary detection and super-node construction.

4.4. Distributed Cost Model

The dominant runtime contributors are shuffle-intensive relational stages and local tensor computation. We model end-to-end cost as:

$$T_{\text{total}} \approx T_{\text{io}} + T_{\text{shuffle}} + T_{\text{compute}} + T_{\text{sync}} \quad (5)$$

For both Branch A (propagation path) and Branch B (decoupled community path), the synchronization term T_{sync} is

minimized: training-critical steps require no iterative cross-worker neighborhood fetches. Executor-side computation is mediated by Spark applyInPandas grouped UDFs with Apache Arrow serialization, reducing JVM-Python transfer overhead while preserving table-native orchestration.

4.5. Complexity Analysis

Phase 0 ingestion performs a single-pass read and Delta write in $O(|V| + |E|)$. Phase 1 LPA executes $O(|E| \cdot t_{\text{iter}})$ edge scans over t_{iter} iterations (typically ≤ 6). Phase 2 boundary detection requires one join followed by a group-by aggregation, costing $O(|E|)$ shuffle plus $O(|V|)$ aggregation. Phase 3.7 propagation performs one join-aggregate per hop, so K hops cost $O(K \cdot |E|)$. Phase 3.8 fits a global Spark ML classifier in $O(|V_{\text{labeled}}| \cdot d \cdot t_{\text{iter}})$ where $d = (K + 1) \cdot d_0$ is the concatenated feature dimension. For Branch B, Phase 3 decomposes into embarrassingly parallel per-community GNN jobs: $O(\sum_{i=1}^m |E_i| \cdot \text{epochs})$ with zero inter-community communication.

5. Runtime System Design on AWS EMR

This section describes the runtime model and failure-mitigation strategies that make EMO stable on commodity EMR clusters.

5.1. Resource Model: JVM Heap, Off-Heap Ten-sors, and Local Disk

Spark executors on YARN have two memory regions (Figure 1): a JVM heap for Spark SQL operations and an

off-heap allocation for native libraries. PyTorch and DGL allocate tensors through native malloc, outside the JVM's accounting. Because YARN enforces container-level memory limits, under-provisioned overhead causes silent container kills even when the JVM heap appears healthy. EMO mitigates this by explicitly configuring memory overhead:

`spark.executor.memory = 28g` `spark.executor.memoryOverhead = 12g`

5.2. Failure Modes and Mitigation

We identify three recurring failure modes in Spark-based GRL deployments and their mitigations. F1 (Off-heap OOM): Native tensor allocations exceed YARN container limits, triggering executor kills. Mitigation: explicit 12 GB memory overhead allocation. F2 (Root-volume exhaustion): EMR worker root volumes (15 GB) are consumed by pip caches, PyTorch build artifacts, and DGL downloads. Mitigation: dynamic environment remapping redirects HOME, TMPDIR, and pip cache paths to the largest attached local EBS volume:

```
os.environ['HOME'] = large_tmp
```

```
os.environ['TMPDIR'] = large_tmp
```

F3 (Library skew): Heterogeneous executor environments (missing packages, version mismatches) cause non-deterministic training failures. Mitigation: bootstrap package synchronization and user-site path patching before task launch.

5.3. Executor Consistency and Reproducibility

EMO synchronizes runtime dependencies across all YARN executors using bootstrap installation and PYTHONPATH patching, ensuring deterministic behavior without pre-imaged cluster nodes. Combined with Delta Lake's versioned artifacts and transaction logs, this provides a reproducible systems contract across repeated runs and cluster restarts.

6. Experimental Evaluation

6.1. Setup and Benchmark Datasets

All experiments run on commodity AWS EMR infrastructure: an 8-node cluster of m5.4xlarge workers (16 vCPUs, 64 GB each) managed by YARN, with a single m5.2xlarge driver node. To ensure comprehensive evaluation across graph scales, we employ a two-tiered benchmark suite: (i) five small-to-medium datasets (WikiCS, Coauthor-Physics, Coauthor-CS, DeezerEurope, Foursquare) establishing algorithmic fidelity and baseline parity against prior GRL and community-based methods, and (ii) three massive-scale graphs (ogbn-products with 2.45M nodes, 123.7M edges; reddit with 233K nodes, 114.6M edges; and ogbn-papers100M with 111M nodes, 3.2B edges) evaluating EMO's distributed AWS EMR cloud execution. GNN architectures include GraphSAGE, GATv2, ARMA, ASAP, and GraphSAGE-CAAN. All Phase 3.7/3.8 scaling experiments use two-hop propagation ($K = 2$), producing concatenated features [x (0)]

||x (1)

||x (2)

].

Figure 3 consolidates the full experimental configuration, including dataset coverage, cluster specifications, fairness controls, and reported metrics. 6.2. Link Prediction and Node Classification

Tables 4 and 5 report link prediction (AUC) and node classification (accuracy) results.

Three key findings emerge from the large-scale evaluations on ogbn-products and reddit.

Link prediction benefits from community structure. On reddit, EMO with Louvain partitioning achieves 92.61% AUC, exceeding the full-graph GraphSAGE baseline (75.52%) by +17.09 percentage points and DistDGL (74.03%) by +18.58 points. On ogbn-products, EMO's LPA-SAGE model reaches 85.29% AUC versus DistDGL's 82.16%. Dense intra-community edge structure, which partitioning preserves, directly benefits link prediction.

CAAN recovers boundary accuracy. Without context recovery, decoupled node classification drops to 52.96% (Louvain) and 72.74% (LPA) on reddit—a loss of up to 44 percentage points. EMO's relational CAAN super-node mechanism recovers this deficit, achieving 92.73% (Louvain-CAAN, +39.77% recovery) and 89.11% (LPA-CAAN, +16.37% recovery), matching within 2% of full-graph models while maintaining zero cross-worker training

communication.

Task-dependent performance profiles. Node classification and link prediction exhibit complementary sensitivity to community decoupling. Link prediction improves because partitioning concentrates positive edges within communities, while node classification degrades because boundary nodes lose inter-community label context. This asymmetry validates the necessity of CAAN: the recovery mechanism closes the classification gap without sacrificing link prediction gains.

6.3. Component-wise Ablation Study

Table 6 isolates the impact of EMO's core subsystems on the reddit graph.

Super-node topology. Removing CAAN context recovery from isolated Louvain GNNs drops accuracy by 39.77 percentage points (from 92.73% to 52.96%), confirming that centroid-based super-node abstraction is essential for mitigating boundary degradation in power-law community graphs.

Delta Lake caching and amortized ingestion. Bypassing Delta transaction log caching and forcing cold S3 reads increases total pipeline latency from 2254.2s to 3842.1s, a 1.7× slowdown from redundant object-store deserialization. Crucially, in enterprise Lakehouse workflows, Phase 0 ingestion (1852.6s) and Phase 1/2 preprocessing are executed once per dataset; subsequent model exploration, hyperpa-

parameter tuning, and downstream training epochs directly reuse the cached Delta tables with zero ingestion overhead, reducing effective per-experiment cost to the fast GNN training phase (208.2s).

Community threshold (K). Sweeping the minor community threshold reveals that finer granularity (K = 100:

92.81%; K = 500: 92.76%) preserves structural context Validation and Evaluation Setup Datasets, hardware, baselines, and protocol used for fair comparison

Datasets Small/medium: WikiCS, Coauthor-Physics, Coauthor-CS, DeezerEurope,

Foursquare Large: reddit, ogbn-products Cluster and Runtime AWS EMR (5 workers), 20 executors, 60 vCPUs Spark + YARN + Delta Lake Executor memory 28g, overhead 12g

Model Families GraphSAGE, GATv2, ARMA, ASAP Community models and CAAN variants

Phase 3.7/3.8 global path

Validation Protocol Fixed train/val/test masks Multiple partition strategies (LPA, Louvain)

Consistent preprocessing contracts Fairness Controls Matched partition bounds and resources Same data source and feature schema Compared under identical cluster

settings Two-Hop Configuration num_hops = 2 for Phase 3.7/3.8 scaling Input

representation: $[x^{(0)} \parallel x^{(1)} \parallel x^{(2)}]$ Classifier trained on cached propagated features

Quality Metrics Node accuracy Link prediction AUC Boundary recovery Systems Metrics

Propagation time Training time End-to-end runtime Scalability Metrics Speedup vs 16

executors Parallel efficiency Coverage consistency Artifacts Tables 3/4/9/12/13 Figure 1

architecture Phase 3.7 scaling plots

Blue boxes: fixed controls used in validation and fairness comparisons. Gray boxes: evaluated dimensions and reported outcomes.

Figure 3. Experimental configuration: dataset coverage, EMR/Spark infrastructure, model families, fairness controls, fixed two-hop configuration for

Phase 3.7/3.8, and reported quality/systems/scalability metrics.

TABLE 4. PERFORMANCE COMPARISON ON LINK PREDICTION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original AUC	Original Time (s)	EMO / ML-GRL AUC	EMO Time (s)	AUC Imp. (%)
WikiCS500K edges	GraphSAGE	0.6517 ± 0.0034	2s ± 10.77	68 ± 0.0012	9s ± 1	19.20%	
WikiCS500K edges	ARMA	0.8634 ± 0.0142	65s ± 30.91	02 ± 0.0003	4s ± 2	5.42%	
WikiCS500K edges	ASAP	0.8773 ± 0.0006	4s ± 80.90	96 ± 0.0013	3s ± 1	3.68%	
Coauthor-Physics4M edges	GraphSAGE	0.6134 ± 0.0123	06s ± 20.65	49 ± 0.0032	5s ± 4	6.77%	
Coauthor-Physics4M edges	ARMA	0.7574 ± 0.0295	81s ± 10.85	89 ± 0.0013	76s ± 20	13.40%	
Coauthor-Physics4M edges	ASAP	0.7489 ± 0.0056	54s ± 190.79	60 ± 0.0005	12s ± 17	6.29%	
Coauthor-CS2M edges	GraphSAGE	0.6172 ± 0.0051	32s ± 10.69	35 ± 0.0011	07s ± 5	12.36%	
Coauthor-CS2M edges	ARMA	0.7442 ± 0.0051	36s ± 40.89	19 ± 0.0011	08s ± 20	19.85%	
Coauthor-CS2M edges	ASAP	0.7578 ± 0.0051	35s ± 50.79	35 ± 0.0051	12s ± 14	4.71%	
DeezerEurope1M edges	GraphSAGE	0.5136 ± 0.0055	9s ± 00.58	21 ± 0.0032	2s ± 1	13.34%	
DeezerEurope1M edges	ARMA	0.6575 ± 0.0079	5s ± 30.73	69 ± 0.0026	8s ± 2	12.08%	
DeezerEurope1M edges	ASAP	0.7920 ± 0.0051	33s ± 60.82	59 ± 0.0016	9s ± 1	4.28%	
Foursquare3M edges	GraphSAGE	0.6497 ± 0.0141	41s ± 120.95	89 ± 0.0019	5s ± 6	47.59%	
Foursquare3M edges	ARMA	0.8100 ± 0.0133	12s ± 100.96	73 ± 0.0002	45s ± 13	19.42%	
Foursquare3M edges	ASAP	0.8392 ± 0.0022	91s ± 40.93	99 ± 0.0022	21s ± 23	12.00%	
ogbn-products123.7M edges	GraphSAGE (LPA)	0.7561455.2s	0.85292196.5s	+12.80%			
ogbn-products123.7M edges	GATv2 (LPA)	0.7561455.2s	0.85292184.7s	+12.80%			
ogbn-products123.7M edges	GraphSAGE (Louvain)	0.7561455.2s	0.8280549.1s	+9.51%			
ogbn-products123.7M edges	GATv2 (Louvain)	0.7561455.2s	0.8282547.9s	+9.53%			

reddit114.6M edgesGraphSAGE (LPA)0.7552315.8s0.7823626.9s+3.59%
 reddit114.6M edges GraphSAGE (Louvain)0.7552315.8s0.9261382.4s+22.63%
 better than aggressive over-clustering (K = 5000: 91.45%)
 at modest additional cost.

6.4. Comparison with DistDGL

Table 7 presents a controlled comparison between EMO and DistDGL under identical partition bounds and cluster resources.

On reddit, EMO's Louvain-CAAN model completes training in 208.2s versus DistDGL's 250.6s, while eliminating 28.66 GB of parameter server memory overhead and

all cross-node network traffic. On ogbn-products, EMO achieves superior link prediction (85.29% vs. 82.16%) and recovers 99.6% of DistDGL's node classification accuracy (71.85% vs. 72.17%) with zero cross-worker communication.

TABLE 5. PERFORMANCE COMPARISON ON NODE CLASSIFICATION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original Acc	Original Time (s)	EMO / ML-GRL Acc	EMO Time (s)	Acc Imp. (%)
---------	-------	------------	--------------	-------------------	------------------	--------------	--------------

WikiCS11K	nodes	GAT	0.6351 ± 0.00756	56s ± 00.8129 ± 0.00910s ± 1	+28.00%	WikiCS11K	
nodes	GraphTransformer	0.5354 ± 0.02845	5s ± 00.7544 ± 0.00716s ± 2	+40.90%	WikiCS11K		
nodes	ClusterSCL	0.8103 ± 0.00051	5s ± 20.8374 ± 0.00121s ± 1	+3.34%			
Coauthor-Physics34K	nodes	GAT	0.7954 ± 0.00811	8s ± 80.9411 ± 0.00398s ± 12	+18.32%		
Coauthor-Physics34K	nodes	GraphTransformer	0.9083 ± 0.00450	8s ± 60.9457 ± 0.002383s ± 8	+4.12%		
Coauthor-Physics34K	nodes	ClusterSCL	0.9436 ± 0.00092	2s ± 130.9644 ±			

0.000391s $\pm$ 17+2.20%

Coauthor-CS18K nodesGAT0.6161 $\pm$ 0.00884s $\pm$ 40.8609 $\pm$ 0.00578s $\pm$ 0+39.73% Coauthor-

CS18K nodesGraphTransformer0.7338 $\pm$ 0.006225s $\pm$ 100.9040 $\pm$ 0.001186s $\pm$ 5+23.20%

Coauthor-CS18K nodesClusterSCL0.9438 $\pm$ 0.002336s $\pm$ 20.9603 $\pm$ 0.000111s $\pm$ 6+1.75%

DeezerEurope28K nodesGAT0.5498 $\pm$ 0.00240s $\pm$ 00.6152 $\pm$ 0.00235s $\pm$ 1+11.90%

DeezerEurope28K nodesGraphTransformer0.6022 $\pm$ 0.00466s $\pm$ 20.6861 $\pm$ 0.00558s $\pm$

1+13.93% DeezerEurope28K nodesClusterSCL0.5725 $\pm$ 0.001181s $\pm$ 40.5933 $\pm$ 0.00149s $\pm$

2+3.63%

ogbn-products2.45M nodesGraphSAGE (LPA)0.7298455.2s0.60762196.5s-16.74% ogbn-

products2.45M nodesGATv2 (LPA)0.7298455.2s0.60742184.7s-16.77% ogbn-

products2.45M nodesGraphSAGE (Louvain)0.7298455.2s0.5311549.1s-27.23% ogbn-

products2.45M nodesGATv2 (Louvain)0.7298455.2s0.5317547.9s-27.14%

reddit233K nodesGraphSAGE (LPA)0.9455315.8s0.7274626.9s-23.07% reddit233K

nodesGraphSAGE (Louvain)0.9455315.8s0.5296382.4s-43.99% reddit233K

nodesGraphSAGE (LPA-CAAN)0.9455315.8s0.89111148.4s-5.75% reddit233K

nodesGraphSAGE (Louvain-CAAN)0.9455315.8s0.9273208.2s-1.92%

TABLE 6. ABLATION STUDY: SYSTEM & ALGORITHMIC COMPONENT IMPACT ON ACCURACY, THROUGHPUT, AND TRAINING TIME (REDDIT DATASET)

Configuration / VariantNode Acc (%) Comm Acc (%) Ingestion Time (s) Partition

Time (s) GNN Train Time (s) Total Time (s)

Full EMO Pipeline (Louvain-SAGE-CAAN)92.73%93.38%1852.6s66.3s208.2s2254.2s EMO

Pipeline (LPA-SAGE-CAAN)89.11%92.02%1852.6s65.4s1148.4s3157.4s

w/o Global Graph (Decoupled Louvain-SAGE)52.96%49.21%1852.6s66.3s382.4s2428.4s

w/o Global Graph (Decoupled LPA-SAGE)72.74%76.37%1852.6s65.4s626.9s2635.9s

w/o Delta Lake Optimizations (Cold S3 Reads)92.73%93.38%3440.5s66.3s208.2s3842.1s

Community Threshold K = 100 (Louvain-CAAN)92.81%93.42%1852.6s66.3s364.5s2410.5s

Community Threshold K = 500 (Louvain-CAAN)	92.76%	93.40%	1852.6s	66.3s	259.1s	2305.1s
Community Threshold K = 5000 (Louvain-CAAN)	91.45%	91.89%	1852.6s	66.3s	134.2s	2120.3s
Full-Graph Baseline (GraphSAGE)	94.55%	—	—	315.8s	315.8s	Distributed GNN Engine
(DistDGL)	94.80%	—	2.0s	250.6s	257.6s	

TABLE 7. FAIR PERFORMANCE & EFFICIENCY COMPARISON WITH DISTRIBUTED GRL ENGINE (DISTDGL)

Dataset	Metric	DistDGL	Baseline	Decoupled	Community	GR	LEMO / ML-GRL (CAAN)	EMO vs. DistDGL Imp.
WikiCS	Accuracy (%)	65.34% ± 0.52	79.97% ± 0.34	81.04% ± 0.24	+15.70%			
	Execution Time (s)	217s ± 018s	218s ± 112.0×	Speedup				
	Network Vol. (GB)	4.8 GB	0.0 GB	0.1 GB	97.9%	Communication	Drop	
ogbn-products	Node Acc (%)	72.17%	60.76% (LPA) / 53.11% (Louv)	71.85% (Louv-CAAN)	recovers 99.6% of accuracy			
	Link AUC (%)	82.16%	85.29% (LPA) / 82.80% (Louv)	85.29%+3.13%	AUC			
	Execution Time (s)	400.5s	1547.9s (Louv)	1547.9s	Communication-Free			
	Network / RAM	46.05 GB	RAM 0.0 GB	Network 0.0 GB	Network Zero	Inter-node Comm.		
reddit	Node Acc (%)	94.80%	52.96% (Louv) / 72.74% (LPA)	92.73% (Louv-CAAN)	recovers 97.8% of accuracy			
	Link AUC (%)	74.03%	92.61% (Louv)	92.61%+18.58%	AUC			
	Execution Time (s)	250.6s	382.4s (Louv)	208.2s (CAAN Train)	1.20×	Faster Training		
	Network / RAM	28.66 GB	RAM 0.0 GB	Network 0.0 GB	Network Zero	Inter-node Comm.		

6.5. Horizontal Scalability

Table 8 and Figure 4 report Phase 3.7/3.8 scaling on the 8-worker cluster under 16, 32, and 64 executor configurations with fixed two-hop propagation. The experiment holds node coverage at 100% and model accuracy constant, vary-

ing only executor parallelism. Figure 4(a) illustrates end-to-end propagation runtime across datasets on a logarithmic scale, while Figure 4(b) depicts observed speedup versus ideal linear scaling.

For ogbn-papers100M, propagation time drops from 2478.3s (16 executors) to 1940.5s (32) and 1892.1s (64), showing meaningful gains up to 32 executors and diminishing returns beyond. For ogbn-products, benefit is modest (84.5s → 80.8s at 32 executors), while WikiCS degrades at higher executor counts due to scheduling overhead exceeding the small graph's compute cost. Across all datasets, Phase 3.8 test accuracy remains constant (WikiCS: 0.7746, ogbn-products: 0.7068, ogbn-papers100M: 0.6327), confirming that this sweep isolates systems scaling from model-quality variation. These results identify 32 executors as the practical operating point for large graphs under CPU-only Spark.

7. Discussion and Future Work

Incremental Time-Travel GRL. Enterprise graphs evolve continuously. Delta Lake's transaction logs enable Time Travel queries (AS OF) that can identify communities with significant structural or feature drift since the last training run, enabling selective retraining of only affected partitions.

Z-Ordering for S3 Locality. Z-Ordering Delta Lake tables by community_id and is_boundary would co-

locate related Parquet files on S3, reducing the number of GET requests when executors read their assigned partitions.

Limitations. Several deliberate design trade-offs merit discussion. (1) Branch B's community-decoupled path requires CAAN super-node recovery for competitive node classification; without it, boundary nodes lose inter-community label context. (2) Louvain partitioning produces higher-quality communities but requires collecting the graph to the driver, limiting it to datasets that fit in driver memory; LPA scales fully in Spark but produces lower-quality cuts. (3) Phase 3.8's global classifier is a linear logistic probe; replacing it with non-linear architectures such as GAMLP [20] may improve accuracy. (4) EMO is currently CPU-only; GPU-accelerated executors would reduce Phase 3 training time at higher infrastructure cost. (5) Scaling experiments show diminishing returns beyond 32 executors, suggesting that future work should investigate GPU-offloaded Spark executors and adaptive repartitioning.

8. Conclusion

We presented EMO, a distributed systems framework that orchestrates graph representation learning natively on transactional data lakes. By expressing community partitioning, boundary detection, and auxiliary graph construction as standard relational operators on Spark SQL over Delta Lake, EMO enables scalable, communication-free GNN training on commodity cloud clusters.

Our evaluation yields three principal findings. First, EMO's CAAN boundary recovery closes the accuracy gap from community decoupling, recovering 97.8% of full-graph accuracy on reddit with zero cross-worker communication.

Second, the Phase 3.7/3.8 propagation path processes ogbn-papers100M (111M nodes, 3.2B edges) with 100% node coverage and consistent accuracy across executor configurations. Third, Delta Lake's transactional semantics provide phase-level resumability and experiment isolation, converting expensive graph preprocessing into deterministic, cacheable stages.

These results demonstrate that transactional data lakes are a viable and practical foundation for large-scale graph representation learning, bridging the gap between enterprise data infrastructure and GNN research.

References

[1]W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 1024–1034.

[2]P. Velickovic,

c, G. Cucurull, A. Casanova, A. Romero, P. Lio, and

Y. Bengio, "Graph Attention Networks," in Proc. International Conference on Learning Representations (ICLR), 2018.

[3]M. Zheng, D. Zheng, R. Tripathy, and G. Karypis, "DistDGL: Dis-

tributed Graph Neural Network Training for Web-Scale Graphs,” in Proc. IEEE High Performance Extreme Computing Conference (HPEC), 2020, pp. 1–8.

[4]. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Gu, and M. Yuan, “Dorylus: Affordable, Scalable GNN Training with Thousands of Serverless Threads,” in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 953–969.

[5]. Mohyuddin, R. Mayer, and H. Jacobsen, “Marius: Accelerating Graph Embeddings on GPUs with Storage-Centric Training,” in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 919–935.

[6] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[7] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, “Two-Level Graph Representation Learning with Community-as-a-Node Graphs,” IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 11, pp. 6602–6616, 2024.

[8] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, “Two-Level Graph Representation Learning Empowered With Subgraph-as-a-Node,” IEEE Transactions on Computers, vol. 73, no. 6, pp. 1502–1516, 2024.

[9] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, “Multi-Level Graph Representation Learning,” in Proc. ACM SIGMOD International Conference on Management of Data, vol. 3, no. 1, Article 56, 2025.

[10] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and

F. Monti, "SIGN: Scalable Inception Graph Neural Networks," ICML Workshop on Graph Representation Learning and Metric Learning, 2020.

[11] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," in Proc. VLDB Endowment, vol. 13, no. 12, 2020, pp. 3411–3424.

[12] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020, pp. 22118–22133.

[13] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008. TABLE 8. PHASE 3.7/3.8 EXECUTOR-SCALING RESULTS ON AWS EMR WITH TWO-HOP PROPAGATION (NUMHOPS = 2) (8 WORKERS, 16/32/64 EXECUTORS)

Dataset	Nodes	Prop. Edges	Executors	Coverage	Prop. (s)	Classifier (s)	Total (s)
WikiCS	11,701	431,206	16	100%	41.69	0.050	61.00×0.7746
WikiCS	11,701	431,206	32	100%	43.77	0.051	30.95×0.7746
WikiCS	11,701	431,206	64	100%	55.39	0.065	10.75×0.7746
ogbn-products	2,449,029	123,718,024	16	100%	84.51	1.195	61.00×0.7068
ogbn-products	2,449,029	123,718,024	32	100%	80.89	0.990	71.05×0.7068
ogbn-products	2,449,029	123,718,024	64	100%	82.31	1.393	61.03×0.7068
ogbn-papers100M	111,059,956	3,228,124,712	16	100%	2478.31	25.126	03.41.00×0.6327
ogbn-papers100M	111,059,956	3,228,124,712	32	100%	1940.57	3.520	14.01.28×0.6327
ogbn-papers100M	111,059,956	3,228,124,712	64	100%	1892.17	1.519	63.61.31×0.6327

64 cores (16 exec) 128 cores (32 exec) 256 cores (64 exec)

Total CPU Cores (Spark Executors) 10 1 10 2 10 3

Execution Time (seconds, log scale) (a) Total Running Time Scaling

ogbn-papers100M (111M nodes)

ogbn-products (2.4M nodes)

WikiCS (11K nodes)

64 cores (16 exec) 128 cores (32 exec) 256 cores (64 exec)

Total CPU Cores (Spark Executors) 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0

Speedup Factor (relative to 16 exec) (b) Speedup Efficiency Comparison

Ideal Linear Speedup

ogbn-papers100M

ogbn-products

WikiCS

Figure 4. Phase 3.7/3.8 CPU-scaling behavior across three graph scale classes. (a) End-to-end propagation runtime on a log scale. (b) Observed speedup versus ideal linear scaling from 16 executors (64 vCPUs) to 64 executors (256 vCPUs).

[14] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical Review E*, vol. 76, no. 3, p. 036106, 2007.

[15] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 8, pp. 736–751, 1998.

[16] S. Brody, S. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[17] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph Neural Networks with Convolutional ARMA Filters," *IEEE Transactions on Pattern Analysis and Machine*

Intelligence, vol. 44, no. 7, pp. 3496– 3507, 2021.

[18] H. Zeng, H. Zhou, A. Srivastava, R. Kanber, and V. Prasanna, “Graph- SAINT: Graph Sampling Based Inductive Learning Method,” in Proc. International Conference on Learning Representations (ICLR), 2020.

[19] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, “Subgraph Federated Learning with Missing Neighbor Generation,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2021, pp. 6671– 6682. [20] W. Zhang, Z. Yin, Z. Sheng, Y. Li, G. Ouyang, X. Li, Y. Yan, and B. Cui, “Graph Attention Multi-Layer Perceptron,” in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2022, pp. 4560–4570.

[21] Q. Huang, H. He, A. Singh, S.-N. Lim, and A. Benson, “Combining Label Propagation and Simple Models Out-Performs Graph Neural Networks,” in Proc. International Conference on Learning Representations (ICLR), 2021.